Programming Assignment 3 (PA3)

Magical Demon Hunters Game
User-Defined Types, Encapsulation, Operator Overloading, Interacting Objects
Out: March 11, 2026 -- DUE: April 7, 2026
EC327 Introduction to Software Engineering – Fall 2026

Total: 200 points

- *You may use any development environment you wish, as long as it is ANSI C++ 11 compatible. Please make sure your code compiles and runs properly under the Linux/Unix environment on the PHO 305/307 (or eng-grid) machines before submitting.*
- *PAs may be submitted up to a week late at the cost of a **30% fixed penalty** (e.g., submitting a day late and a week late is equivalent). It is in your best interest to complete as many checkpoints as possible before the deadline. If you have missing implementations in your original submission, you may complete and submit the missing solutions during the following week. Any submissions after the deadline will be subject to the 30% penalty. No credit will be given to solutions submitted after the 1-week late submission period following the deadline.*
- *Follow the assignment submission guidelines in this document or you will lose points.*

Submission Format (Must Read)

- Use the **exact** file names specified in each problem for your solutions.
- Push your changes to the repository generated by GitHub Classroom.
- Complete submissions should have **22 files.**
- Please do **NOT** submit *.exe and *.o or any other files that are not required by the problem.
- **Code must compile in order to be graded. Otherwise, this is an automatic zero.**
- Comment your code (good practice!). We **may** use your comments when grading.

Coding Style (reminder from PA2) As you become an experienced programmer, you will start to realize the importance of good programming style. There are many coding "guidelines" out there and it is important that you begin to adopt your own. Naming conventions will help you recognize variable names, functions, constants, classes etc. Similarly, there are many ways to elegantly format your code so that it is easy to read. All of these issues do not affect compilation and hence are easy to overlook at this stage in your development as a programmer. However, your ability (or inability) to create clean, readable code could be the difference in your career when you leave college.

Good reference: https://google.github.io/styleguide/cppguide.html If you find any other good references, feel free to post the link on Piazza!

Class Specifications

Each class and its members are described below by listing its name, the prototypes for the member functions, and the names and types of the member variables. You **MUST** use the prototypes, types, and names in your program as specified here, in the same upper/lower case. Function argument names are allowed to be different. **Failure to follow these specifications will result in lost points.**

Checkpoint 4

The Model-View-Controller (MVC) Pattern

Model

(15 points)

The `Model` is a central component in the MVC pattern and stores all game objects in memory. Hence, it contains various arrays of pointers to the instances of the `GameObject` class. Also it offers multiple methods to interact with the `Controller` and `View` components. Here, it has the following structure:

Private members:

- `int time;`
  - the simulation time.

We have a set of arrays of pointers and a variable for the number in each array:

- `GameObject * object_ptrs[10];`
- `int num_objects;`
- `Mage * mage_ptrs[10];`
- `int num_mages;`
- `ManaSpire * spire_ptrs[10];`
- `int num_spires;`
- `DemonHideout * hideout_ptrs[10];`
- `int num_hideouts;`

Each object will have a pointer in the `object_ptrs` array and also in the appropriate other array. For example, a `Mage` object will have a pointer in the `object_ptrs` array and in the `mage_ptrs` array.

Public members:

- `Model();`
  - It initializes the time to 0 and then creates the objects using **new**, and stores the pointers to them in the arrays as follows:
  - The list shows the object type, its id number, initial location, and subscript in `object_ptrs`, and the subscript in the other array.
    * Mage 1 (5, 1), `object_ptrs[0]`, `mage_ptrs[0]`
    * Mage 2 (10, 1), `object_ptrs[1]`, `mage_ptrs[1]`
    * ManaSpire 1 (1, 20), `object_ptrs[2]`, `spire_ptrs[0]`
    * ManaSpire 2 (10, 20), `object_ptrs[3]`, `spire_ptrs[1]`
    * DemonHideout 1 (0, 0), `object_ptrs[4]`, `hideout_ptrs[0]`
    * DemonHideout 2 (5, 5), `object_ptrs[5]`, `hideout_ptrs[1]`
  - Here is a description of the objects that should exist.
  - S1 is a ManaSpire
    * ID number is 1
    * initial location is (1, 20)
    * crystal cost is 1
    * crystal capacity is 100
  - S2 is a ManaSpire
    * ID number is 2
    * initial location is (10, 20)
    * crystal cost is 2
    * crystal capacity is 200
  - M1 is a Mage
    * name is Serena
    * ID number is 1
    * initial location is (5, 1)
    * speed is 1
  - M2 is a Mage
    * name is Rumi
    * ID number is 2

    * initial location is (10, 1)
    * speed is 2
  - D1 is a DemonHideout
    * ID number is 1
    * location is (0, 0)
    * mana cost is 1
    * GP cost is 2
    * experience per battle is 3
    * battles is 10
  - D2 is a DemonHideout
    * ID number is 2
    * location is (5, 5)
    * mana cost is 5
    * GP cost is 7.5
    * experience per battle is 4
    * battles is 20
  - Set `num_objects` to 6, `num_mages` to 2, `num_spires` to 2, `num_hideout` to 2;
  - Finally, output a message: **"Model default constructed"**

- `~Model();`
  - the destructor deletes each object, and outputs a message: **"Model destructed."**

Note: For purposes of demonstration, add to `GameObject` a destructor function that does nothing except output a message: **"GameObject destructed."** Define similar ones for `Mage`, `ManaSpire`, and `DemonHideout`. This is so that you can see the order of destructor calls for an object.

There are three functions that provide a lookup and validation services to the main program (`Controller`). They return a pointer to the object identified by an id number, depending on the type of object we are interested in. The functions search the appropriate array for an object matching the supplied id, and either return the pointer if found, or 0 if not.

- `Mage * GetMagePtr(int id);`
- `ManaSpire * GetManaSpirePtr(int id);`
- `DemonHideout * GetDemonHideoutPtr(int id);`
- `bool Update()`
  - It provides a service to the main program. It increments the time, and iterates through the `object_ptrs` array and calls `Update()` for each object. Since `GameObject::Update()` will be made virtual, this will update each object.

  - returns `true` if any one of the `GameObject::Update()` calls returned true.
  - If the player finishes all the `DemonHideouts` the game should print **"GAME OVER: You win! All battles done!"**. The game should then exit. **Try using the exit function to achieve this.**
  - If **all** the `Mages` are knocked out and can't move, print **"GAME OVER: You lose! All of your Mages' mana is lost!"**. The game should then exit.

- `void Display(View& view);`
  - Provides a service to the main program. It outputs the time, and generates the view display for all of the `GameObjects`.
  - This will be created later, comment out for now.
- `void ShowStatus();`
  - It outputs the time and outputs the status of all of the `GameObjects` by calling their `ShowStatus()` function.

Implement the polymorphism in the class hierarchy as follows:

- Declare `bool Update()` to be a pure virtual function in `GameObject`. This makes `GameObject` an abstract base class and ensures that each of the derived classes will have defined an update function, or we get a linker error to tell us we have left it out.
- Declare `ShowStatus()` to be virtual in `GameObject`.
- Declare `ShouldBeVisible()` to be pure virtual function in `GameObject`.
- Important: Make the destructors in `GameObject` and `Mage` virtual.

Your `main` function and its sub functions can now be radically simplified because they are no longer responsible for keeping track of all the objects or how many of them or what kinds there are. Remove all the declarations of `Mages`, `ManaSpires`, and `DemonHideouts` from `main`, and replace them with declaring a single `Model` object.

GameCommand

(15 points)

The GameCommand represents the Controller of the MVC pattern and provides multiple functions that interpret user input in order to perform the appropriate actions.

You should create a set of functions that can be called from main to handle the processing of user provided commands. The command functions should use the `Model` member functions like `GetManaSpirePtr()` to check that an input id number is valid and get a pointer for the object if it

is. The `DoAdvanceCommand()` and `DoRunCommand()` functions can just call `Model::Update()` to update all the objects, and `Model::Display()` can be called to display the current view of the game. Note that some commands will cause messages to be printed. Each command function should be given the Model object (by reference). The function prototypes are listed below along with messages they print:

- `void DoMoveCommand(Model & model, int mage_id, Point2D p1);`
  - If the command arguments are valid prints **"Moving (Mage name) to (p1)"**
  - otherwise prints **"Error: Please enter a valid command!"**
- `void DoMoveToSpireCommand(Model & model, int mage_id, int spire_id);`
  - If the command arguments are valid prints **"Moving (Mage name) to Mana Spire (spire_id)"**
  - otherwise prints **"Error: Please enter a valid command!"**
- `void DoMoveToHideoutCommand(Model & model, int mage_id, int hideout_id);`
  - If the command arguments are valid prints **"Moving (Mage name) to hideout (hideout_id)"**
  - otherwise prints **"Error: Please enter a valid command!"**
- `void DoStopCommand(Model & model, int mage_id);`
  - If the command arguments are valid prints **"Stopping (Mage name)"**
  - otherwise prints **"Error: Please enter a valid command!"**
- `void DoBattleCommand(Model & model, int mage_id, unsigned int battles);`
  - If the command arguments are valid prints **"(Mage name) is battling"**
  - otherwise prints **"Error: Please enter a valid command!"**
- `void DoRecoverInSpireCommand(Model& model, int mage_id, unsigned int crystals_needed);`
  - If the command arguments are valid prints **"Recovering (Mages name)'s mana"**
  - otherwise prints **"Error: Please enter a valid command!"**
- `void DoAdvanceCommand(Model& model, View& view);`
  - prints **"Advancing one tick."**
- `void DoRunCommand(Model& model, View& view);`

  - prints **“Advancing to next event.”**

Note that a command is valid if all arguments are valid. For example, a Mage ID argument is valid if there exists a Mage in the game with that ID.

Move all these `Do**Command` functions into a separate `.h` and `.cpp` file.

Your program should work as before. You should be able to command the Mages to move around, battle in the Hideout, recover mana at the Spire and list the status of all the objects. When the program terminates, you should see the correct sequence of destructor messages.

Here is a description of the commands and their input values:

- **m** ID x y
  - “move”: command Mage ID to move to location (x, y)
- **s** ID1 ID2
  - “move towards a ManaSpire”: command Mage ID1 to start heading to ManaSpire ID2.
- **d** ID1 ID2
  - **“**move towards a DemonHideout”: command Mage ID1 to start heading towards DemonHideout ID2.
- **h** ID
  - “stop”: command Mage ID to stop doing whatever it is doing
- **c** ID crystal_amount
  - **“**buy crystals”: command Mage ID to buy `crystal_amount` of crystal at a ManaSpire.
- **b** ID battle_amount
  - **“**complete `battle_amount` battles at a DemonHideout”: command Mage ID to complete `battle_amount` of battles at a DemonHideout.
- **a**
  - “advance”: advance one-time step by updating each object once.
- **r**
  - “run”: advance one-time step and update each object, and repeat until either the update function returns true for at least one of the objects, or 5 time steps have been done.
- **q**
  - “quit”: terminate the program

You do NOT need to do error checking. This will be done in PA4.

You must have a separate command-handling function for each command that collects the input required for the command and calls the appropriate object member functions. We recommend using the switch statement to pick out the function for each command; this is the simplest and cleanest way to do this sort of program branching. For example, to handle the "move" command, the case would look like this:

```
case 'm':
    DoMoveCommand(m1, m2);
    break;
```

All input for this program must be done with the stream input operator >>.

***Important**: Your program must "echo" each command to confirm it and to provide a record in the output of what the input command was. For example, if the user enters "m 1 10 15" the program should output something like "moving 1 to (10, 15)". This way, if output redirection is used to record the behavior of the program, the output file will contain a record of the input. If there is an error in the input, there should be an informative error message, but your program is not responsible for trying to output an exact copy of the erroneous input. So the echo does not have to be present or complete if there is an error in the input.*

CHECKPOINT IV

Implement the Model and the GameCommand and add them to your program. You now should be able to actively have a user manually enter commands and have them play your game. Write a TestCheckpoint4.cpp file and in the main function command the Mages to move to DemonHideout, buy crystals, etc., and list the status of all game objects. Make sure you comment out parts that reference View as that does not exist yet. When the program terminates, you should see the correct sequence of destructor messages.

DO NOT MOVE ON UNTIL YOU ARE 100% SURE YOU UNDERSTAND THIS CHECKPOINT

If you do not make sure that these Model and GameCommands work fully, the rest of your game will NOT work. Use this time to also start to modify the provided makefile to also work for testing Checkpoint 4. DON'T RUSH THROUGH THIS.